个人财务分析系统项目讲解

按简历简介与五项技术点展开的面试准备材料

依据所提供简历与当前项目代码整理 2026 年 10 月 28 日

本文围绕简历中的个人财务分析系统，说明为什么需要这个项目、实现中需要处理哪些难点，以及每项技术解决了什么问题。正文采用第一人称口述方式，方便先理解再用自己的话回答。具体场景示例用于说明设计，不表示发生过真实用户事故。

项目主线是让财务分析结果可核对。模型理解问题和安排步骤，确定性工具负责账单计算与异常规则，检索负责提供财务资料依据，服务层负责保存会话和展示过程。这些层次分别验证，避免只凭回答是否流畅判断系统效果。

阅读说明：原简历的评测数字混用了不同实验，本稿按原始记录复核后更正。代码中已实现的能力、评测使用的能力和后续扩展会在对应位置说明。本文是项目讲解材料，不替换原始简历，也不把开发样例成绩描述为线上效果。

简历对应项	本文重点
项目简介	财务业务问题 确定性计算 可核对的证据链
Agent Runtime	多轮循环 执行边界 消息恢复 Skill 子 Agent MCP
多格式入库	PDF PPTX 图片 OCR 缓存 原件与来源定位
知识检索	分块 父子映射 向量 BM25 RRF 重排与实际接入状态
评测优化	题库 指标 消融 真实结果与结论边界
交互服务	FastAPI SSE 并发 中断 确认 回放与恢复

第一部分 项目简介与财务业务问题

开场可以怎样介绍

“我做的是一个面向个人账单和财务资料的分析 Agent，主要支持收支查询、疑似重复扣款和订阅涨价检查，以及基于资料的知识问答。比如用户问九月份为什么花费增加，系统需要先确定时间范围，再查询账单、按类目比较，必要时找出异常交易，最后结合工具结果解释原因。”

“这个项目的设计重点是结果可核对。我把金额统计和异常规则放在确定性代码中执行，模型负责理解问题、选择工具和组织回答。知识问答保留原文来源，账单分析保留筛选条件、金额和交易编号；评测再分别检查工具计算、检索证据和 Agent 行为。这样出现错误时能追到具体环节，而不只知道最终回答不对。”

“实现上，我复用 LangChain 的模型、工具和消息抽象，自行编写多轮执行循环；资料侧接入多种文件格式和混合检索；交互侧通过 FastAPI 与 SSE 展示工具过程，并处理确认、中断和刷新恢复。当前是可复现的个人项目，尚未建设多用户企业平台。”

为什么需要确定性计算

问题与难点：财务结果可能语言上很合理，数字和统计口径却是错的。例如支出与退款符号相反、日期边界遗漏、未分类交易被当成没有消费，都会产生难以察觉的偏差。直接让模型阅读全部账单并心算，也难以复现计算过程。

我的做法：“我统一交易模型，金额从字符串转换成 Decimal，明确支出为正、收入或退款为负。查询工具接收日期、账户、类目、商户、聚合方式与分组方式，再由确定性查询函数计算；模型得到的是条件、命中笔数、汇总和分组结果，而不是自行拼出的金额。”

具体例子：假设一组账单中有两笔 128.00 元支出和一笔 −28.00 元退款，净支出应为 228.00 元。这个例子说明合计的口径，不代表系统能够仅凭负号判断每笔款项究竟是工资还是退款。真实展示时还要说明时间范围、账户和是否包含未分类交易。

进一步处理：遇到“上个月”这样的相对时间，工具描述要求先检查数据覆盖范围和当前日期；查询日期两端都包含。完全没有分类结果时，按类目查询会明确拒绝给出貌似正常的零，而是提示改按商户查询。部分交易未分类时，分类统计还需要解释覆盖范围。

财务异常为什么采用规则

问题与难点：同一个交易编号重复出现，可能是数据重复导入；不同编号、同商户、相近日期、相同金额，才可能是实际重复扣款。如果把这些情况都在入口简单去重，就可能吞掉需要检查的交易。

我的做法：“我把导入一致性与业务异常分开。后端对重复文件和账户加交易编号做导入幂等；不同编号的同额交易继续保留。异常工具再按商户、金额和时间窗识别疑似重复扣款，默认时间窗为三天，并返回交易编号和日期供用户核对。”

“对异常大额，我使用同商户或同类目的四分位间距围栏，默认阈值为第三四分位数加三倍四分位间距，样本太少的组不判断。这样检查的是与同组主体相距多远，而不是固定把金额最高的一部分交易标成异常。”

“对订阅，我结合出现次数、金额取值数量和扣款间隔规律识别周期性支出，再检查涨价。对于用户可能忘记取消的订阅，系统列出周期扣款并展示按当前价格折算的年化金额，交由用户判断是否仍需要。”

效果与边界：这些规则能返回可复现的候选发现，不等于自动认定欺诈或错误。年化金额是按当前价格折算的未来支出，与过去一年实际发生额不同。模型可以解释发现，最终判断仍需要结合原始账单。

一次完整问题如何经过系统

以“九月份支出为什么增加，有没有重复扣款”为示例。前端提交问题，Runtime 先让模型识别任务；Agent 调用数据概览和交易查询，得到月份范围、总额与分组对比，再调用异常工具查明细。如果需要解释某条资料中的规则，再调用知识检索。最终答案引用已经返回的数值、条件和依据。

如果需要纠正商户分类，执行会进入确认等待；用户允许后才调用写操作。每一步以事件展示，完成轮的消息和日志持久化。刷新页面回放已发生的事件，服务重启恢复完成轮上下文；未完成轮按中断处理。

第二部分 Agent Runtime

简历对应内容为自研多轮工具执行循环、执行边界、失败恢复、Skill 三级渐进披露、子 Agent 上下文隔离和 MCP 子进程接入。

多轮工具执行循环

问题与难点：一个分析问题往往需要多个有依赖的步骤，前一步结果会影响下一步工具和参数。一次固定模型调用无法完成这种反馈循环；同时，工具异常或中断容易留下不完整的消息历史。

我的做法：“我复用 LangChain 的模型客户端、工具声明和消息结构，把工具描述绑定给模型，再自己实现执行循环。模型返回工具名称和参数，我执行工具，把结果作为 ToolMessage 写回上下文，然后让模型继续判断，直到给出最终答案或达到停止条件。”

实现细节：一条模型回复可能同时包含多个工具调用。每个 tool_call 都必须有对应的工具结果。工具执行失败时写入错误结果；用户停止后，尚未执行的调用补上未执行结果，避免下一轮携带缺少配对消息的历史。工具开始、结束、结果和答案通过事件回调传递给终端或 Web；展示回调出错时不会改写工具执行事实。

可以怎样验证：对模型返回多工具调用、某个工具抛异常、用户中断和展示回调异常分别构造测试，检查调用与结果仍然配对、下一轮能够继续。这里验证的是状态和协议完整性，没有证明自研循环比框架现成执行器更快。

步数限制和输出截断

问题与难点：模型可能反复查询相近问题，增加费用和等待时间；账单分组和明细输出过长，也会不断扩大上下文。

我的做法：“我为主 Agent 设置单轮最多二十四个模型循环步骤，子 Agent 最多十步。超过预算时说明未完成，让用户缩小范围。工具端优先做业务裁剪，例如最多展示四十组分组结果、二十条异常明细；主要只读工具出口再对超过二万字符的文本做截断，并说明原始长度。”

为什么这样做：先控制业务结果比任意截掉后半段更容易解释。比如分组数量超限时提示还有多少组未列出，模型就能继续缩小日期或商户范围；用户也能知道当前展示不是全部数据。

需要说准确：二十四限制的是循环步骤，一步可以包含多次工具调用，不能说成最多调用二十四次工具。二万字符是主要工具出口使用的限制，并非 Runtime 对任意外部工具输出都做了全局拦截；字符数量也不能直接换算为精确 token 成本。这些值是保护参数，尚未通过压测证明最优。

上下文溢出与失败恢复

问题与难点：长对话叠加工具结果后可能超过模型窗口；一次失败如果污染了会话消息，用户后续无法继续。另一类风险是自动重试整轮，把已经完成的写操作再次执行。

我的做法：“我在每轮开始时记录消息列表检查点。模型请求超出上下文或发生异常，就删除检查点之后本轮新增的消息，保留此前的完成历史，返回可读错误。用户可以缩小查询范围或重新开始会话。”

回滚范围：这是消息回滚。假如先前工具已经完成分类修改，之后模型调用失败，删除消息不会撤销数据库更新，也不会撤销已经写入的长期记忆。数据库事务与幂等约束负责各自的业务一致性，因此我不会用无条件重试整轮 Agent 来处理所有错误。

恢复范围：当前 Web 在完成一轮时，把消息快照、解除 busy 状态和结束事件放进同一事务；进程重启后恢复已完成轮的上下文。尚未完成的轮次标为中断并释放占用，不自动重放工具。这支持继续会话，不等于任意工具步骤都能断点续跑。

Skill 三级渐进披露

问题与难点：月报、预算检查和对账有不同的业务流程。如果把所有流程和参考资料都加入 system prompt，每轮模型请求都会携带大量无关内容。

我的做法："我把 Skill 内容分成三级。第一级只展示名称和简短说明，供模型判断有没有合适能力；第二级通过 use_skill 读取 SKILL.md 正文，获得流程；第三级再按需要读取 references 或 scripts 中的资源。"

实现细节：一级清单有一千五百字符预算，超出预算按整条省略，不切掉半句说明。三级资源用相对路径定位，在路径解析后检查仍位于当前 Skill 内，防止目录穿越或符号链接跳出目录。单个资源返回上限为六万字符。读取 scripts 在这里表示读取脚本文本，不会自动执行。

具体例子：用户只问某月总支出，通常直接查询即可；用户要求月度复盘时，再读取 monthly-report 流程，需要报告结构时进一步读取参考模板。减少的是进入模型上下文的无关正文，并非 Skill 发现阶段完全不读取磁盘。

效果与边界：按需披露让能力扩展与每轮提示词大小不必完全同步增长。但清单预算不足时，某些 Skill 不进入目录，会降低可发现性；现有代码也没有给出独立的 token 节省比例。

子 Agent 上下文隔离

问题与难点：某个子问题可能要探索很多次，主会话最终却只需要结论。如果把所有中间结果都留在主会话，之后每次模型调用都要继续承担这些内容。

我的做法："我把聚焦的分析任务交给独立 Session。子 Agent 只接收主 Agent 写出的完整任务，不继承父会话消息。它的查询过程保留在自己的消息列表里，结束后向主 Agent 返回结论和上下文用量记录。"

权限边界：子 Agent 采用五个只读工具白名单，包括数据概览、交易查询、异常检测、订阅清单和知识检索。它没有分类修改、写记忆、再次委派或 MCP 接入能力。这样新增一个主 Agent 写工具时，不会自动扩大子 Agent 权限。

具体例子：主 Agent 让子 Agent 分析指定账户、指定三个月内的餐饮趋势，必须把时间、账户和统计口径写清楚。上下文隔离会同时隔离背景信息，任务描述不完整时，子 Agent 就可能无法正确分析。

效果与边界：当前委派同步执行，收益主要是主会话减少中间探索内容，没有实现多个子任务并行调度。记录的子上下文字符数和回传字符数可以辅助判断委派价值，不能直接宣称等量的 token 或费用下降。

MCP 子进程接入

问题与难点：工具能力可能由独立服务提供。直接把所有工具代码塞进 Agent 会增加耦合；子进程通信还要处理协议输出污染、超时、调用配对和退出清理。

我的做法："我实现了项目需要的 MCP stdio 客户端与账单服务端。启动子进程后进行初始化，再通过 tools/list 发现名称、描述和参数 schema，适配成 LangChain 工具；执行时发送 tools/call。当前服务暴露账户清单、商户清单和交易明细三个工具，并统一加 ledger__ 前缀防止重名。"

实现细节：stdout 用于 JSON-RPC 通信，调试信息走 stderr；请求使用锁进行串行配对，并设置超时和输出大小限制。不同 Web 数据库上下文使用对应客户端，子进程读取同一数据库事实，避免主服务已更新账单而 MCP 仍读取旧 CSV。子进程仅接收运行所需环境和明确传入的数据库配置，不默认继承模型密钥。

效果与边界：接入失败时保留内置工具并给出警告，关闭时释放管道和子进程。当前实现覆盖项目需要的 stdio 工具发现和调用，没有实现完整 MCP SDK 或 Streamable HTTP。进程边界也不等于操作系统安全沙箱。

Runtime 常见追问

为什么不直接使用现成 Agent："我保留了 LangChain 的模型、消息和工具抽象，自己写循环是为了明确控制停止条件、消息补齐、事件展示和失败检查点。这是控制能力的取舍，没有性能对比支持它更快。"

子 Agent 和 MCP 的隔离有什么不同："子 Agent 隔离推理历史和工具集，仍在当前程序内运行；MCP 对接独立进程中的工具服务。两者控制的边界不同，都需要单独考虑权限。"

发生错误后为什么不自动再跑一遍："上一轮可能已经写过数据库或记忆。先区分可重试的远程读取和可能有副作用的业务步骤，再由对应幂等键与事务保护，不能把消息回滚当作业务撤销。"

第三部分 多格式知识入库

简历对应内容为使用 PyMuPDF、python-pptx 和本地 OCR 接入 PDF、PPTX、图片及 Markdown，处理扫描内容、表格和备注，并保留原件与来源位置。

为什么把不同文件统一成知识块

问题与难点：个人财务资料可能来自电子文件、扫描件、培训幻灯片和截图。如果要求用户先转成 Markdown，使用成本高，也容易丢失页码、表格对应关系和备注中的条件。

我的做法："我先为每种格式选合适的解析器，把输出统一为正文、来源位置和提取方式，再进入切块和索引。PDF 先用 PyMuPDF 读文字层，PPTX 用 python-pptx 读结构，扫描内容和图片使用 RapidOCR。这样文档提取可以独立测试，后续检索不用为每种格式单独实现一套逻辑。"

为什么没有全部交给视觉模型：当前目标是抽取可检索文字，能读取原生文本时就优先读取，扫描内容再做本地识别。本地 OCR 不需要为每张图片额外请求视觉接口；但后续向量化和问答仍可能调用外部服务，不能把整个系统说成完全离线。

PDF 文字层和扫描页怎样处理

问题与难点：有些 PDF 可直接选中文字，有些整页是图片，还有一些正文能提取、关键数字却在图片里。只抽文字会漏内容，全部 OCR 又会增加耗时和识别误差。

我的做法："我提供 auto、always 和 off 三种 OCR 模式。auto 先读取页面原生文字；没有文字或页面存在图片时，以一百五十 DPI 渲染整页进行 OCR。纯文字页直接使用原生提取结果；图文混排页合并文字层和 OCR，只去掉与原生文字完全对应的 OCR 重复行。"

来源与边界：每页保留页码和 native、ocr 或 native+ocr 提取标记，方便定位错误来自解析还是检索。PDF 当前输出是文本，尚未实现任意复杂表格的结构恢复或跨页表格拼接。简历中的表格提取需要解释为 PPTX 结构化表格可按行列读取，PDF 主要依靠文字层和 OCR 获取文本。

PPTX 表格 备注与图片怎样处理

问题与难点：幻灯片中一个文本框可能只有"预算"，另一个只有金额。如果分别过滤短文本或分别切块，就会丢掉标签或让数值失去上下文。重要内容还可能藏在组合形状、备注或图片中。

我的做法：“我按幻灯片汇集文字，递归遍历组合形状；表格按行读取，单元格之间以制表符分隔；备注单独提取；内嵌图片再做 OCR。每一部分保留幻灯片序号、页标题和位置标签，例如第三页的表格或备注。”

兼容边界：旧版 .ppt 需要独立 LibreOffice 进程先转为 .pptx，再复用解析流程；缺少程序时明确提示另存为 .pptx。目前 Docker 镜像没有包含 LibreOffice。解析能保留表格行列文本和备注，但不承诺恢复任意图表关系、动画顺序和复杂空间布局。

内容指纹缓存为什么不能只用文件名

问题与难点：OCR 耗时，重建索引不应反复识别同一资料。仅按文件名缓存会重复处理改名文件，也可能把同名新内容误判为旧内容；文件大小和修改时间也不足以准确表示内容是否变化。

我的做法：“我把原始内容、扩展名、解析方案版本、OCR 模式和依赖版本共同计算成指纹。指纹相同复用提取结果，内容或配置变化就重新解析；缓存损坏也会重新提取。这样既减少重复计算，也避免解析器更新后继续使用旧结果。”

需要区分：解析缓存与上传去重是两件事。当前同名文件拒绝覆盖，不同文件名但相同内容的上传，并没有统一禁止。缓存说明计算可以复用，不表示系统已经实现跨文件名的全局资料去重。

原件保存和失败处理

我的做法：“入口校验文件名、格式、空文件和二十 MB 大小限制。文件先进入暂存区，解析成功后通过同文件系统硬链接发布原件；目标存在就失败，避免并发覆盖。如果索引构建失败，撤销本次原件发布。检索块继续保存文件相对路径、页或幻灯片位置、标题和提取方式。”

具体例子：用户核对一条报销规则时，可以从来源记录回到某份 PDF 的对应页，或 PPTX 的对应幻灯片。当前定位粒度是资料和页面位置，没有实现页面坐标框精确高亮。Markdown 入库会规范 UTF-8 文本，其他格式保留原始文件字节。

多格式入库常见追问

怎样判断上传成功不是空壳：“我检查提取文本、产生的知识块以及来源位置是否对应。测试覆盖文字页、扫描页、图文混排、PPTX 表格和备注、缓存复用与失效、损坏和加密文件，以及越界文件名；接口返回成功只是其中一层。”

OCR 错了怎么办：“OCR 结果是提取文本，不是真值。系统保留原件和提取方式，方便核对；对于金额、条款数字和低清晰度扫描件，仍需人工确认。当前没有统一完成低置信度复核工作流，也不能说识别准确率已经达到某个数值。”

第四部分 知识检索

简历对应内容为中文递归切分、父子块映射、text-embedding-v4、BM25、RRF 和 gte-rerank-v2。需要区分主工具实际执行路径与已实现的实验模块。

父子块怎样平衡召回粒度和上下文

问题与难点：块太大，关键条件可能被其他内容稀释；块太小，规则结论与适用条件会分离。假设只召回“按固定标准扣除”，模型并不知道它属于哪一类事项。

我的做法：“我先按标题层级构建父块，保存完整标题路径，过长正文再按自然段拆分，父块上限约九百字符。非 Markdown 内容先按页或幻灯片位置组织。向量侧把父块进一步切成默认最多二百字符的子块，按空行、换行和中文标点逐级递归切分，最后才硬切。”

“检索先命中较细的子块，再按 parent_id 映射到父块并去重，最终向模型返回对应父块的上下文。这里的完整指对应父块，不是整份文件，也不保证规则跨越多个父块时自动找齐。”

效果与边界：父子策略提供召回和展示两种粒度，但原固定 Markdown 语料本来较短，现有数据没有证明该策略单独带来显著提升。面试可以解释设计动机与机制，不应补造其独立收益。

向量化和 Chroma 怎样保持版本一致

问题与难点：正文更新后，旧向量若仍指向新内容，会形成检索与原文不一致；切块策略或 embedding 模型变化后，复用旧索引也可能造成混用。

我的做法：“父块正文保存在 index.json，Chroma 主要保存子块向量和标识。父块标识包含来源、标题路径与正文哈希；向量索引指纹包含语料、子块方案和模型。指纹一致复用向量，并重新建立父子映射；不一致就重建，避免向量与正文版本漂移。”

“默认 embedding 是 text-embedding-v4，每批最多十条文本，并按接口响应中的 index 对齐回原输入。模型调用和向量存储抽象分开，持久化可以使用 Chroma，测试和已有小语料评测也可以使用内存余弦存储。”

需要说准确：Chroma 是当前支持的持久化后端，但下文指标来自内存余弦评测，不能把它们描述为 Chroma 性能测试、线上响应速度或端到端回答正确率。

BM25 为什么与向量一起保留

问题与难点：有些问题直接包含术语、标题或数字，适合词法匹配；有些问题使用日常表达，和原文没有相同关键词，需要语义召回。两种错误来源不同，因此需要有办法单独比较。

我的做法："词法侧分别为标题路径和正文建立 BM25 索引；中文采用二元字符组加单字并过滤部分停用词，字母和数字保留整词。向量侧用问题向量检索子块，归并到父块。简历所说的字面与语义双路，是两类方法的概括；代码融合时实际有标题 BM25、正文 BM25 和语义三个通道。"

具体例子：直接询问某个文档术语时，词法检索能利用准确字面；询问"工资没涨，为何后几个月到手变少"时，语义通道有机会关联到累计预扣相关内容。此例说明查询形式差异，不代表当前系统能保证每个真实问题都正确识别。

RRF 为什么按排名融合

问题与难点：BM25 分数与向量相似度不在同一个尺度，直接相加容易让某一路数值范围主导排序。

我的做法："我采用 RRF，依据候选在每一路中的名次计算贡献，再把各路贡献相加。当前参数 k 为六十，每路先取三十个候选。一个候选如果在多路排名都靠前，融合分数会更高，这样不用先把不同模型的原始分数校准到同一尺度。"

实现边界：RRF 不能判断哪条候选真能回答问题，也不保证增加一路就一定更好。标题与正文可能对同一类词法信息重复投票，弱相关候选可能因此压过语义通道的强结果。这个解释是待进一步消融验证的可能原因，不是已经完成的加权优化。

重排解决的是候选顺序

问题与难点：融合后正确依据可能已经进入候选池，却排在最终返回范围之外。如果只看最终 Top5，容易把排序错误误认为召回失败。

我的做法："我实现 gte-rerank-v2 重排模块，把问题与候选的来源、标题路径和正文一起送去评分，先对较大的候选池重排，再取最终结果，默认候选上限二十。模块失败时返回原融合顺序，相关性评分不能当成答案正确概率。"

当前接入状态：重排模块和评测入口已实现，但主 Agent 的 search_knowledge 当前调用的是融合检索，默认返回四个块，并没有实际串接重排。准确口述应是实现了可比较的混合召回与重排模块，并在评测链路验证重排；主工具接入还需补齐。评测 Top5 与主工具默认 Top4 也应明确区分。

缓存和远程依赖失败怎样处理

我的做法：“embedding 和重排有按文本、问题与模型派生的缓存，重复内容可以复用结果。查询阶段语义通道失败时，融合层仍可返回 BM25 结果；重排失败时维持原顺序。可选的重试和熔断模块进一步减少短暂故障对请求的影响。”

边界：首次创建或更新向量库的 embedding 失败，目前没有统一包在同样的 BM25 降级边界里，不能把局部查询降级概括为任何向量故障都不影响系统。熔断也只能保护可选依赖，主模型连接失败时仍需要明确报错。

检索常见追问

重排能补回没召回的资料吗：“不能。重排只处理已有候选，所以我要分别看候选召回和最终排序。正确证据没有进池时，需要检查解析、切块、query 或召回配置。”

混合检索一定比纯向量好吗：“不能预先这样判断。保留词法、向量、融合与重排，是为了比较不同问题和故障条件下的取舍。现有开发集上纯向量更好，我会如实报告，再用更多独立题目决定默认方案。”

第五部分 评测优化与指标解释

本部分对应简历的评测优化条目。原简历中的十八题、融合优于纯向量以及两组提升数字不属于同一个实验，因此先明确题库和实验范围，再解释可支持的结果。

为什么要分层评测

问题与难点：最终回答错误，可能来自文档没有提取、切块失去条件、检索漏证据、排序靠后、工具选错，也可能是模型看到依据后仍理解错误。只凭回答流畅或抽查几个案例，无法定位问题。

我的做法：“我把确定性查询、异常检测、检索证据和 Agent 行为分开验证。计算与规则可以独立用结构化结果对比；检索先检查必需原文是否进入结果；工具路由再检查模型有没有调用合适工具；最终回答单独复核数字、依据和表述。这样优化有明确对象。”

一百题框架覆盖哪些情况

“我整理了一百题分层框架，覆盖字面、转述、多证据、无答案和工具路由。每题保留问题、参考要点与题族，有证据的题目额外保留来源文件、小节和原文锚点；按题族和证据小节划分开发与测试，避免同一知识点的不同问法跨集合，产生内容泄漏。”

题型	题量	检查目标
字面题	30	原文关键词和术语匹配
转述题	30	日常表达对应的语义召回
多证据题	15	回答条件所需依据是否找齐
无答案题	15	证据不足时的回答边界
工具路由题	10	账单查询与知识问答等工具选择

数据状态：整体划分为六十道开发题和四十道测试题。当前题目、参考要点和证据标注仍是待人工复核的草案，不能说成全部经过人工标注的金标准，也不能把一百题框架规模当成本次实验实际测了一百道正向题。

Recall 和 MRR 分别说明什么

证据 Recall@5：“先看一道题需要多少段必需依据，再计算前五条结果覆盖了多少。例如需要两段依据，只找到一段，这道题为百分之五十；重复返回同一段不会增加证据覆盖。最后对所有有答案题求平均，因此报告的是题均宏平均。”

MRR@10：“看第一条相关证据排在第几位。第一位命中记一，第二位记二分之一，前十位都没有则记零，再对题目求平均。它说明相关证据是否靠前，但多证据题即使第一位命中，也不等于找齐所有依据。”

统计边界：至少找到一条依据的命中率、必需证据覆盖率和找齐全部依据的题目比例是不同指标，不能混用。无答案题不进入正向 Recall 分母，工具路由另算。网络失败保留在对应实验分母中按零计，避免只报告成功请求。

怎样做同条件消融

“我固定语料和问题，分别运行 BM25、纯向量、RRF 融合、融合后重排，记录候选与最终结果，再逐题定位证据在哪一步丢失。相同语料比较不同方案，能避免把语料扩大带来的变化误认成算法收益。”

本次实验范围：十二篇 Markdown、六十个父块、八十三个向量子块；四十五道不重复正向开发查询，其中十八道字面、十八道转述、九道多证据。每种方案重复三轮，即一百三十五次正向尝试。重复运行用于观察波动，没有增加独立问题数量。

后端和分母：这次向量实验采用内存余弦，在全部子块上检索并归并父块。无答案题另行保留，工具路由不在这次正向表中；后来新增的 PDF、PPTX 和图片也不属于这份固定语料。重排的四次正向失败按零计入平均。

方案	题均证据 Recall@5	MRR@10	正向失败
标题与正文 BM25 融合	75.6%	0.611	0/135
纯向量	98.9%	0.875	0/135
BM25 与向量 RRF 融合	83.3%	0.744	0/135
RRF 融合后增加重排	92.6%	0.848	4/135

哪个提升可以如实讲

“在这组固定语料开发样例里，我对同一融合链路增加 gte-rerank-v2 后，题均证据 Recall@5 从百分之八十三点三提高到九十二点六，增加九点三个百分点；MRR@10 从零点七四四提高到零点八四八。这支持重排改善了融合候选的排序，但纯向量在相同数据上的 Recall 仍为百分之九十八点九，所以我没有证据说融合重排已经最优。”

原简历混用在哪里：83.3% 到 92.6% 属于四十五道开发查询的 RRF 到 RRF 加重排；MRR 0.370 到 0.780 则属于十八道转述题的 BM25 到纯向量，对应 Recall 为 55.6% 到 100%。这几项不能组合成十八题上 RRF 融合优于纯向量的结论。

参考答案：“纯向量更好是当前结果，不会因为我已经实现了混合检索就修改结论。我保留多种可比较方案，下一步用更大、独立且人工复核的查询集，连同延迟、远程调用成本和术语题表现决定默认策略。字段合并、加权融合或按问题类型路由都是待验证方向，目前不把它们说成已实现收益。”

建议替换的简历表述

“搭建覆盖字面、转述、多证据、无答案及工具路由的一百题分层评测框架，结合证据 Recall@5、MRR@10 与逐题消融定位召回和排序问题；在固定语料的四十五道开发样例中，对 BM25 与向量融合链

路引入 gte-rerank-v2 重排，将题均证据 Recall@5 由 83.3% 提升至 92.6%，MRR@10 由 0.744 提升至 0.848。”

口述时紧接着说明，这是待人工复核题库上的开发集结论，纯向量同组更高。它不代表独立测试集、生产 Chroma 性能、真实用户回答准确率或高并发吞吐。

评测常见追问

一百题是不是样本仍然太少：“是，尤其同题族问法相关，题目还没有完成独立人工复核。当前框架适合定位链路问题，需要继续补充真实需求、独立题族与人工审查，才能扩大结论范围。”

三轮是不是一百三十五道题：“不是。只有四十五个不重复查询，同一问题重复三次仍不是三个独立样本。报告要同时给出题量、尝试次数、失败数和语料范围。”

MRR 能不能仅凭保存的 Top5 重新计算：“不能完整恢复第六到第十位。这次 MRR 是按原始逐题指标字段汇总；保存的 final_hits 只有前五条，因此报告中会保留这个证据边界，不能说仅凭前五条重建了 Top10 排名。”

第六部分 FastAPI 与 SSE 交互服务

简历对应内容为工具过程实时展示、当前轮中断、跨 HTTP 权限确认，以及刷新后的事件回放和续播。

为什么把执行过程做成事件

问题与难点：一次分析可能有多次模型往返和工具调用。如果只在全部完成后返回，用户无法区分正在查询、等待确认、出现故障或已经停止。耗时同步调用还不能直接阻塞异步请求处理。

我的做法：“我把 Runtime 行为转换为步骤、工具调用、工具结果、确认请求和最终答案等事件。FastAPI 接收发送问题等操作并提交后台任务，浏览器通过独立 SSE 连接接收过程；同步模型和工具工作在线程池中执行。”

SSE 机制：服务器返回 text/event-stream，持续发送结构化事件，设置不缓存和关闭代理缓冲的响应头，空闲时每十五秒发送心跳。用户提交消息或确认，仍然走普通 HTTP 接口。SSE 是基于 HTTP 的流式响应，不能表述成 SSE 与 HTTP 是两种互斥协议。

效果与边界：现在能展示开始了哪一步、调用了什么工具及最终完整答案，尚未逐 token 输出模型文本。能说进度更可见、用户可以及时操作，但没有实测依据宣称降低了模型计算时间或提升了多少并发能力。

固定线程池和容量控制

问题与难点：如果每个请求都创建线程，模型变慢或用户一直不确认时，会不断占用资源。后台执行本身不能自动解决并发容量问题。

我的做法："我采用固定线程池，默认同时运行两轮分析，另外允许八个任务等待。超出容量返回 429；同一个会话上一轮没结束，再次发送返回 409。默认每个会话每分钟最多六轮，避免单会话持续刷请求。"

效果与边界：限制的是当前单进程的执行资源，不能当成已压测的 QPS 指标。确认等待也占用分析线程，多个长期待确认任务会降低可用容量。当前限流按会话且存于进程内，共享访问口令也没有提供逐用户费用配额。

跨 HTTP 权限确认如何对应原来的任务

问题与难点：分类修改等操作需要等待用户同意，而用户点击按钮会产生另一条 HTTP 请求。系统必须把这次决定交回正确的待执行操作，避免历史按钮放行新操作。

我的做法："每次确认生成独立 confirmation_id。后台分析线程先发送确认请求事件，再等待 threading.Event；前端提交选择与编号，服务端核对它仍是当前待确认项、尚未回答且会话有效，然后唤醒原来的分析线程。"

等待边界：等待发生在线程池里的分析线程，不阻塞 FastAPI 异步事件循环；默认三百秒超时按拒绝处理。重复点击和过期编号拒绝放行；停止或关闭会话时，也会拒绝并唤醒等待。

具体例子：第一次请求纠正某商户类别，用户还未决定就停止本轮；随后新一轮又请求另一项修改。第一次的 confirmation_id 已过期，旧按钮不能授权第二次操作。这里是一次写操作的交互授权，尚不能替代多用户身份与角色权限。

中断为什么不是立即杀掉请求

问题与难点：同步模型请求或已经进入的工具，未必能被安全地立刻取消；强制结束也不能自动撤回已经完成的财务修改。

我的做法："用户点击停止后设置停止标志，并释放待确认等待。Agent 在后续检查点结束，不再执行剩余工具；未执行调用补齐结果消息。接口返回正在停止，下一轮再清除标志，用户可以继续提问。"

边界：正在进行的同步模型请求仍可能等到返回或超时；已经完成的数据库修改不会撤销。浏览器关闭 SSE 连接也不会自动取消任务，主动停止需要调用停止接口。分析生命周期和事件连接生命周期分别管理。

刷新回放和断线续播

问题与难点：只把事件放在内存或浏览器队列里，刷新会丢历史，慢客户端也可能让队列无限增长；单有事件编号却没有对应历史，仍无法恢复页面。

我的做法："我把事件持久化到数据库，每个会话通过数据库更新语句递增分配 seq，并在同一事务写入事件。SSE 把 seq 放进 id 字段；连接断开后，浏览器重连带 Last-Event-ID，服务端查询其后的事件，前端再按序号去重。"

完整刷新：页面先清空当前展示，从头回放历史事件，再持续接收新事件。普通断线重连则从已经收到的序号继续。两个过程解决的场景不同，不能只保存游标却丢掉游标之前的界面内容。

慢客户端：持久模式的内存通知队列容量为一，只用来唤醒连接读取数据库，事件正文不靠它保存。队列已满时可以合并唤醒；之后仍从数据库按序补读，不会因为跳过一次通知就丢掉对应事件。

服务重启后恢复到什么程度

我的做法："一轮完成时，消息快照、释放 busy 和 turn_end 事件一起提交。服务重启后恢复已完成轮的模型上下文和事件；上次仍在运行的轮次标记中断并释放占用，不自动重新执行工具。"

部署约束：运行线程、确认对象和部分限流状态仍在内存，启动时的未完成轮恢复也依赖单 API 模式，因此当前不能直接增加多个 Uvicorn worker。要扩容，需要明确任务所有权、确认分发和持久步骤状态，而不只是增加进程数。

交互服务常见追问

为什么用 SSE："当前主要需要服务器向浏览器推送进度，用户操作可以单独走 HTTP。SSE 的单向事件流符合这个需求，也有事件编号和重连机制；不需要为了流式展示就引入双向长连接协议。"

有数据库为什么还需要通知队列："数据库保存完整历史，队列用于及时唤醒读取。这样不用为每个慢客户端积累完整历史队列，也不依赖高频空轮询。"

刷新恢复和业务重新执行有何关系："回放只是重新展示已经发生的事件，不调用工具。恢复完成轮上下文是继续对话的基础，尚未完成轮不会自动重做。展示幂等和写操作幂等分别处理。"

第七部分 当前后端扩展与部署边界

本部分是当前仓库相较所提供简历的补充，不属于原简历的五条主要工作。面试只有在被问到后端可靠性或后续扩展时再展开，避免把后加内容混成原有评测结论。

关系数据库与后台文档任务

“Web 账单、会话、事件和任务进入关系数据库，开发默认 SQLite，也支持 PostgreSQL 和 Alembic 迁移。金额在 PostgreSQL 使用两位小数 NUMERIC，在 SQLite 保存整数分，领域计算仍用 Decimal。导入通过文件指纹和账户加交易编号控制幂等，内容冲突整批回滚；分类纠正通过版本号检查并与前后值审计共同提交。”

“OCR 和索引重建可能耗时，所以上传返回任务编号，由独立 worker 处理。数据库任务通过条件更新领取、租约续期和令牌校验防止过期执行者写状态；知识目录用文件锁保护索引修改。成功、失败、重试和取消可查询，但文件副作用与数据库状态不是一个跨资源原子事务，也没有实现原生 OCR 的强制中止。”

重试与熔断

“短暂连接故障和限流可以做有界重试，参数错误和密钥错误应尽早返回。主模型保留 SDK 层一次重试，不重跑整个 Agent；embedding 与重排有各自重试和进程内熔断状态，连续失败后暂时跳过远程依赖，冷却后允许探测。”

“当前策略保护的是局部远程依赖，没有统一实现整轮总 deadline，也不跨进程共享。重试次数、超时与费用要一起考虑，不能用无限重试掩盖配置错误，或对已提交的财务写操作盲目重试。”

Kafka 财务事件处理

“账单写入后，汇总和异常分析可以独立更新。我把新增交易和 Outbox 待发事件放在同一数据库事务；发布器收到 Kafka 确认后再标记已发布，broker 不可用时账单仍可保存，事件保留等待重试。”

“汇总与异常消费者使用独立消费组，把去重记录与结果更新放在同一数据库事务，提交后再提交消费位点。因此消息可能重复投递，但重复消费不会再次累计同一财务结果。这是至少一次交付配合业务幂等，不是跨数据库和 Kafka 的全局恰好一次事务。”

“Kafka 按账户作为 key，顺序保证限于分区。账单导入成功不代表派生结果已更新；当前单 broker 是演示配置，旧事件重放还受保留期与数据库去重状态约束。历史 backfill 只补缺失事件，不是完整投影重建工具。”

Docker 和企业部署

“我提供同一应用镜像和 Compose，统一启动数据库、迁移、单 API 和文档 worker，可选叠加 Kafka。应用以固定非 root 用户运行，数据使用命名卷持久化，并设置健康检查和依赖顺序；模型凭证由运行环境传入。”

“配置检查和本地独立进程验证，不能等同于镜像在生产运行。本稿没有新增 Docker 容器构建与运行验收，也没有远程 CI 成功记录。当前共享 token、统一账单和知识库适合受控演示；正式多用户使用还需要身份、资源归属、操作人审计、备份与监控。”

第八部分 面试讲述顺序与核对索引

三分钟怎样组织

先用半分钟说明用户问题和可核对的设计主线，再用一分钟讲 Agent 如何根据反馈调用确定性工具；接着用半分钟讲多格式资料如何保留来源并进入检索；用半分钟给出同条件消融和真实结论；最后用半分钟说明 SSE 事件、确认和恢复。面试官对某一环节感兴趣，再展开对应章节的机制与边界。

不要逐项背诵全部技术栈。每讲一项技术，都先给一个具体问题，再说明选择、实现和验证。例如讲子 Agent 时先解释主会话被探索细节占满，讲父子块时先解释粒度与上下文的矛盾，讲 RRF 时先解释不同分数不在同一尺度。

必须解释准确的表述

容易混淆的说法	本项目准确解释
上下文回滚等于撤销操作	只回滚消息 已提交业务写入另由事务与幂等处理
二十四步等于二十四次工具调用	限制模型循环步骤 一步可包含多个调用

容易混淆的说法	本项目准确解释
子 Agent 已实现并行调度	当前同步委派 主要隔离推理历史与工具权限
所有 PDF 表格已结构化	PDF 提取文字与 OCR PPTX 表格按行列读取
主工具每次都会重排	当前主工具使用融合 重排接入评测链路
混合方案已优于纯向量	现有开发集纯向量更高 重排改善融合自身
一百题全部参与该指标	该表为四十五道正向开发查询 三轮重复
SSE 已实现逐字生成	当前推送执行事件和最终完整答案
刷新续播等于工具断点续跑	回放已有事件 未完成轮不自动重放工具
Kafka 与 Docker 证明企业高可用	当前单节点演示 仍需身份隔离和生产运维

代码和验证记录在哪里

主题	核对入口
交易模型与查询	fa/models.py fa/ingest.py fa/query.py fa/tools/query.py
异常规则	fa/anomalies.py fa/tools/anomaly.py tests/test_anomalies.py
Runtime 与技能	fa/agent.py fa/config.py fa/skills.py fa/prompt.py tests/test_agent.py
子 Agent 与 MCP	fa/workflow.py fa/mcp/client.py mcp_server/ledger_server.py tests/test_mcp.py
多格式入库	fa/retrieval/documents.py fa/knowledge.py tests/test_documents.py
检索与重排	fa/retrieval/hybrid.py children.py dense.py rerank.py fa/tools/knowledge.py
评测设计与复核	eval/benchmark_cases.py eval/benchmark.py eval/benchmark_audit.md tests/test_benchmark.py
会话与事件恢复	web/server.py web/sessions.py backend/session_store.py tests/test_persistent_sessions.py
后端与容器	BACKEND.md DOCKER.md backend/jobs.py backend/outbox.py compose.yaml